

Second-Order Methods

Newton, Gauss-Newton & the trust dial

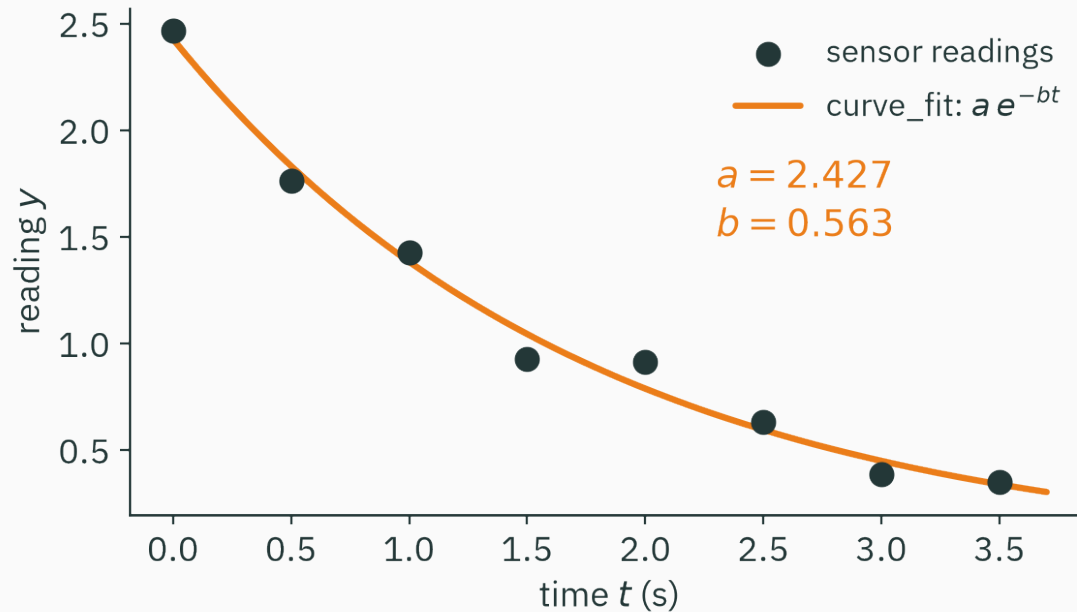
Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

The hook: inside scipy's curve fitter

A sensor, a decay, eight points

You're calibrating a sensor. After a pulse, its reading decays like $a e^{-bt}$ — and the datasheet needs the decay rate b . You log eight noisy readings:



Two unknowns (a, b), eight equations that don't quite agree. Every lab on earth solves this the same way...

Everyone calls the same one-liner

```
from scipy.optimize import curve_fit

def model(t, a, b):
    return a * np.exp(-b * t)

popt, pcov = curve_fit(model, t, y, p0=(1.0, 0.2))
# popt = [2.4274, 0.5628]
```

No learning rate. No 10,000 iterations. It lands on the answer in **about six steps**.

The docs say `method='lm'` — **Levenberg–Marquardt**. What is that? By the end of today you will have built it, piece by piece, and these exact numbers will fall out.

Today, in one sentence

L17's gradient descent trusted a **line** — and paid for it in steps, with the condition number κ setting the price.

If a linear model buys one step, a **quadratic model buys a jump straight to the bottom of its bowl.**

- **Newton**: model the loss with L9's quadratic Taylor, jump to the model's minimum.
- **Gauss-Newton**: when the loss is a **sum of squares**, get the curvature almost free.
- **Levenberg-Marquardt**: a trust dial between the two worlds — the thing inside `curve_fit`.

Learning outcomes

By the end of this lecture you will be able to:

1. Derive ★ the Newton step $\delta = -H^{-1}\nabla f$ by minimizing the quadratic model.
2. Explain why Newton finishes **any** quadratic in one step — for **any** condition number.
3. Run Newton by hand in 1-D and recognize **quadratic convergence** (digits double).
4. Price a Newton step (d^2 memory, d^3 solve) and name its failure modes (saddles, ascent).
5. Set up **nonlinear least squares**: residual vector r , its Jacobian J , $\nabla \mathcal{L} = J^\top r$.
6. Derive the **Gauss-Newton** system $(J^\top J)\delta = -J^\top r$ and read LM's λ dial.
7. Say precisely what `scipy.optimize.curve_fit` runs.

A parabola instead of a line

L17 in one slide: the line and its price

Gradient descent came from trusting the **first-order** Taylor model for one small step:

$$f(\mathbf{x} + \boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \boldsymbol{\delta} \quad \Rightarrow \quad \boldsymbol{\delta} = -\eta \nabla f(\mathbf{x})$$

- The learning rate η = how far you trust a line that is only true at a point.
- On a bowl with condition number κ : the safe η is set by the **steepest** wall, so the **flattest** direction crawls — the step budget grows like κ . That was L17's punchline.

A line has no bottom. However far you trust it, it only ever says “downhill is that way” — never “stop **here**”.

L9's gift: the model with a bottom

L9 built the **second-order** Taylor model and promised it would run Module 4:

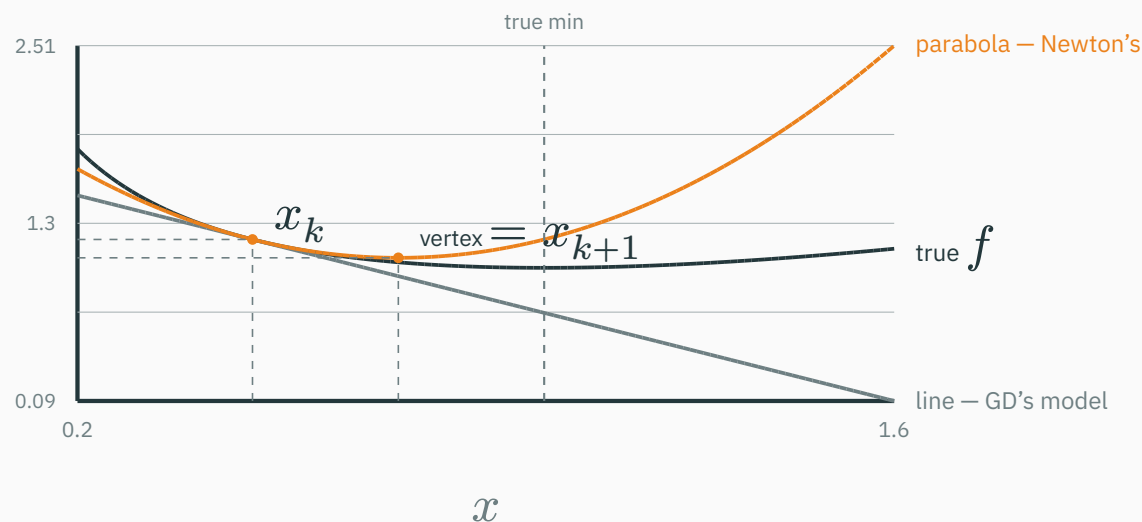
$$f(x + \delta) \approx \underbrace{f(x)}_{\text{height}} + \underbrace{\nabla f(x)^\top \delta}_{\text{tilt}} + \underbrace{\frac{1}{2} \delta^\top H(x) \delta}_{\text{bend}}$$

- GD kept height + tilt. Today we keep the **bend** too.
- If H is positive definite (L9: all eigenvalues > 0), this model is a **bowl** — and a bowl has a **bottom you can name**.

New policy: don't step along the model — jump to the model's minimum.

Picture first: two models, two policies

Same function, same point, both local stories drawn at $x_k = 0.5$ (here $f(x) = x - \ln x$):



the line never bottoms out — **you** must choose a step; the parabola has a vertex — **it** chooses the step for you

The two policies, side by side

	gradient descent (L17)	Newton (today)
local model	line: $f + \nabla f^\top \delta$	bowl: $f + \nabla f^\top \delta + \frac{1}{2} \delta^\top H \delta$
the step	$-\eta \nabla f$ — direction from the model, length from you	jump to the model's minimum — no η anywhere
uses	slopes only	slopes and curvature

One question remains: **where exactly is the bowl's bottom?** That formula is today's .

The Newton step

★ Step 1 — in 1-D: minimize the parabola

D DERIVATION

The model around x_k , as a function of the step δ :

$$q(\delta) = f(x_k) + f'(x_k) \delta + \frac{1}{2} f''(x_k) \delta^2$$

A parabola in δ — minimize it like any 1-D function (set $\frac{dq}{d\delta} = 0$, L7):

$$f'(x_k) + f''(x_k) \delta = 0 \quad \Rightarrow \quad \delta^* = -\frac{f'(x_k)}{f''(x_k)}$$

Numbers — $f(x) = x - \ln x$ at $x_k = 0.5$: $f' = 1 - \frac{1}{0.5} = -1$, $f'' = \frac{1}{0.5^2} = 4$:

$$\delta^* = -\frac{-1}{4} = 0.25 \quad \Rightarrow \quad x_{k+1} = 0.75 \quad \text{— the vertex in the picture, exactly}$$

★ Step 2 — in n -D: same move, matrix clothes

D DERIVATION

$$q(\delta) = f(x_k) + \nabla f^\top \delta + \frac{1}{2} \delta^\top H \delta$$

Differentiate with respect to δ using L9's two gradient rules ($\nabla(g^\top \delta) = g$, and the ★ rule $\nabla(\frac{1}{2} \delta^\top H \delta) = H\delta$ for symmetric H):

$$\nabla_\delta q = \nabla f + H\delta = 0$$

The Newton system: $H\delta = -\nabla f$, so $\delta_{\text{Newton}} = -H^{-1} \nabla f$

Fine print: this stationary point is the model's **minimum** only when H is positive definite — L9's bowl test. Pin that; it becomes a plot twist in Section 5.

Read the step like an engineer

$$\underbrace{\delta = -\eta \nabla f}_{\text{GD}} \quad \text{vs} \quad \underbrace{\delta = -H^{-1} \nabla f}_{\text{Newton}}$$

- Newton is GD with the scalar η promoted to a **matrix-valued learning rate** H^{-1} .
- In the eigenbasis (L5): direction i gets step $1/\lambda_i$ — steep walls small steps, flat valleys big ones. Exactly what L17 begged for, **per direction, automatically**.
- In practice you never form H^{-1} : **solve** $H\delta = -\nabla f$ (Module 1 discipline — solving is cheaper and more stable than inverting).

Numbers: one Newton step on L9's bowl

$f(x, y) = x^2 + xy + y^2$ at $(1, 1)$ — the running function of L9, with its old friends:

$$\nabla f = \begin{pmatrix} 2x + y \\ x + 2y \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \text{ (everywhere)}$$

Solve $H\delta = -\nabla f$ by hand (2×2 elimination):

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \delta = \begin{pmatrix} -3 \\ -3 \end{pmatrix} \Rightarrow \delta = \begin{pmatrix} -1 \\ -1 \end{pmatrix} \Rightarrow x_1 = (0, 0)$$

And $\nabla f(0, 0) = \mathbf{0}$ — the **exact minimum, in one step**. (Chalkdust's solver agrees, live: $\delta = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$.) Luck? Next slide: it can't miss.

On any quadratic, Newton cannot miss

Take **any** quadratic $f(x) = \frac{1}{2}x^\top Ax + b^\top x + c$ with A positive definite. Then (L9 rules):

$$\nabla f = Ax + b, \quad H = A \quad \text{— the same matrix at every point}$$

So from **any** starting point x_0 :

$$x_1 = x_0 - A^{-1}(Ax_0 + b) = x_0 - x_0 - A^{-1}b = -A^{-1}b$$

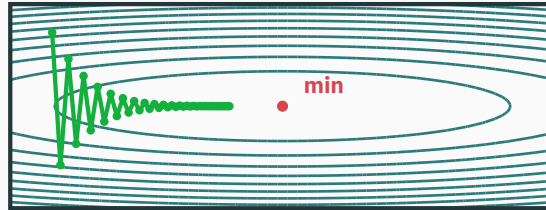
— which is exactly where $\nabla f = 0$: **the** minimum.

On a quadratic, the model **is the function — one jump, done, and κ never entered the formula.**

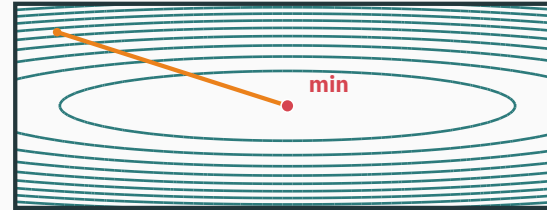
The $\kappa = 50$ bowl: crawl vs teleport

L17's torture chamber, $f = \frac{1}{2}(x^2 + 50y^2)$, same start — both trajectories computed live:

gradient descent · 40 steps



Newton · 1 step



GD: zigzags die out, then a long crawl — 40 steps and still $\sim 23\%$ of the x -gap left. Newton: $H\delta = -\nabla f$, solved once.

Crank κ from 50 to 5000: GD's step budget grows $\sim 100 \times$ (L17). Newton's budget: still **one**. Its step is built from H , so conditioning is priced in.

Code: Newton in five lines

```
def newton(grad, hess, x, steps=10):
    for _ in range(steps):
        x = x - np.linalg.solve(hess(x), grad(x))    # solve, never invert
    return x

A = np.array([[1., 0.], [0., 50.]])                # the kappa = 50 bowl
newton(lambda x: A @ x, lambda x: A, np.array([-2.2, 0.65]), steps=1)
# array([0., 0.])                                <- one step, exact
```

Compare L17's tuning ritual: no $1r=$, no schedule, no divergence watch. The price appears later — first, let's enjoy it on functions that **aren't** parabolas.

Newton's method is applied to $f(x) = 3(x - 2)^2$ starting at $x_0 = 10$. Where is x_1 ?

A. $x_1 = 2$

B. $x_1 = 6$ — halfway

C. $x_1 = 4$

D. Cannot say — depends on the learning rate

Correct: A — $x_1 = 2$, the exact minimum

Why: f is a quadratic, so Newton's parabola **is** f : $f'(10) = 6(10 - 2) = 48$, $f'' = 6$, $\delta = -\frac{48}{6} = -8$, $x_1 = 2$. One jump, done. And option D is a trap worth savoring: Newton has **no learning rate** — the curvature f'' already decides how far a given slope should be trusted.

Newton beyond quadratics

Not a parabola? Re-model and jump again

Real losses aren't quadratics — the model is only a **local rumor**. So iterate:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k) \quad (1\text{-D: } x_{k+1} = x_k - f'(x_k)/f''(x_k))$$

Each jump lands at the **model's** bottom, not the function's — then fits a fresh parabola there.

Our 1-D running example $f(x) = x - \ln x$ (min at $x^* = 1$): the update simplifies beautifully —

$$x_{k+1} = x_k - \frac{1 - 1/x_k}{1/x_k^2} = x_k - (x_k^2 - x_k) = 2x_k - x_k^2$$

No calculus left at run time. Let's run it.

You've met Newton before

Newton's method was born for **root finding** — solve $g(x) = 0$ by following the tangent:

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} \quad (\text{Newton-Raphson, the school version: } \sqrt{2} \text{ via } x_{k+1} = (x_k + 2/x_k)/2)$$

Minimizing f means solving $\nabla f = 0$. Substitute $g = f'$:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)} \quad \text{— the same formula we just derived from the parabola}$$

Two readings, one method: **jump to the parabola's vertex** $\equiv$ **chase a zero of the slope**. The second reading explains the dangers coming in Section 5: a zero of ∇f need not be a minimum...

Watch the digits double

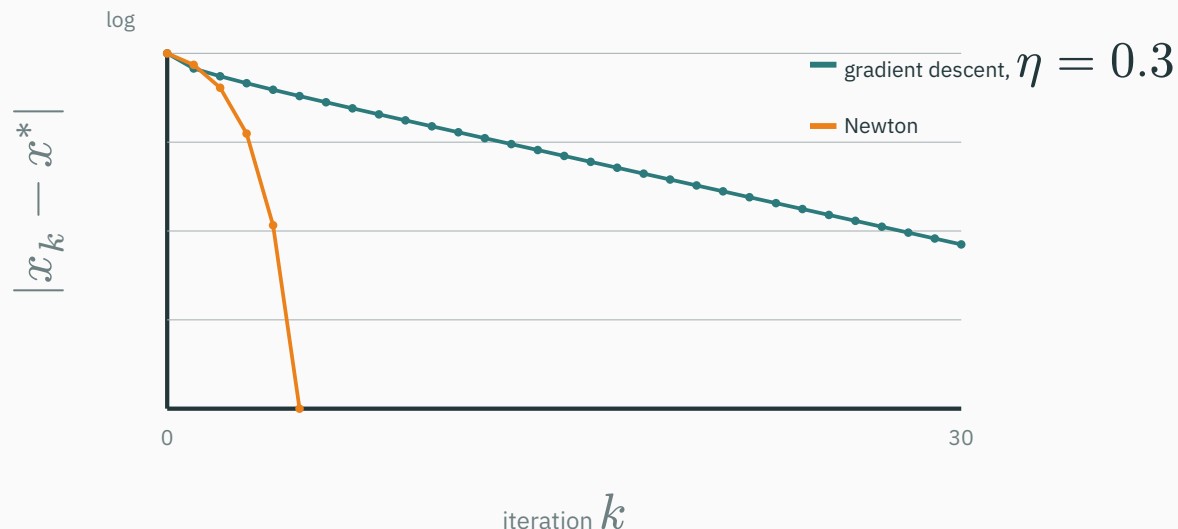
$x_{k+1} = 2x_k - x_k^2$ from $x_0 = 0.5$, run live in the slide (target $x^* = 1$):

k	x_k	error $x_k - 1$	correct digits ($-\log_{10}$)
0	0.5000000000	5.0×10^{-1}	0.3
1	0.7500000000	2.5×10^{-1}	0.6
2	0.9375000000	6.3×10^{-2}	1.2
3	0.9960937500	3.9×10^{-3}	2.4
4	0.9999847412	1.5×10^{-5}	4.8
5	0.9999999998	2.3×10^{-10}	9.6

The correct-digit count $0.3 \rightarrow 0.6 \rightarrow 1.2 \rightarrow 2.4 \rightarrow 4.8 \rightarrow 9.6$ — **doubling every step**.

Here it's exact: $x_{k+1} - 1 = 2x_k - x_k^2 - 1 = -(x_k - 1)^2$, so $e_{k+1} = e_k^2$. The error **squares**.

Same function, same start: GD ($\eta = 0.3$) vs Newton, error per iteration — log scale:



GD is a **straight line** on log paper — a constant factor (0.7) per step. Newton's slope itself steepens: that's the error squaring. 10 digits by $k = 5$ vs 5 digits by $k = 30$.

Quadratic convergence, said honestly

Near a minimum with H positive definite and smooth curvature:

$$\|e_{k+1}\| \leq C\|e_k\|^2 \quad \text{— "quadratic convergence": digits double per step}$$

- GD's **linear** convergence multiplies the error by a constant (L17: $1 - \frac{1}{\kappa}$, painful for large κ); Newton **squares** it — and squaring doesn't care about κ .
- The catch is the word **near**: far from the minimum the parabola is a rumor about a place it's never been. No global promise.

Newton is a **finisher**, not an explorer: started close, it's unbeatable; started far, it can jump anywhere — including uphill. The next section is the bill and the crash reel.

The bill and the dangers

The bill: what a step costs at AI scale

For d parameters, per iteration:

	needs	memory	arithmetic
gradient descent	∇f	$O(d)$	$O(d)$ once ∇f is in hand
Newton	$\nabla f, H$, a solve	$O(d^2)$	$O(d^3)$ for the solve

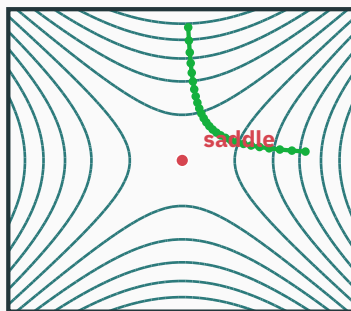
At L9's modest network, $d = 10^6$: H has 10^{12} entries $\approx$ **4 TB** in float32 (L9's warning, now due) — and one $O(d^3) = 10^{18}$ -FLOP solve is **hours on a modern GPU, for a single step**.

Newton's genius is per-step; its cost is per-dimension, cubed. In 2-D it's free. At 10^6 -D it's fantasy.

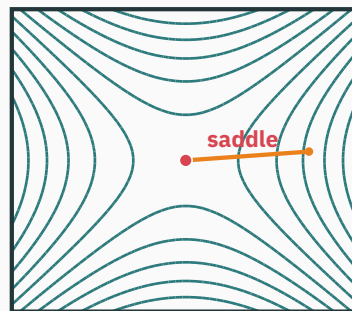
Danger 1: saddles pull it in

Newton chases $\nabla f = 0$ — and (L16) a saddle **is** a $\nabla f = 0$ point. On $f = \frac{1}{2}(x^2 - y^2)$:

gradient descent · drifts off



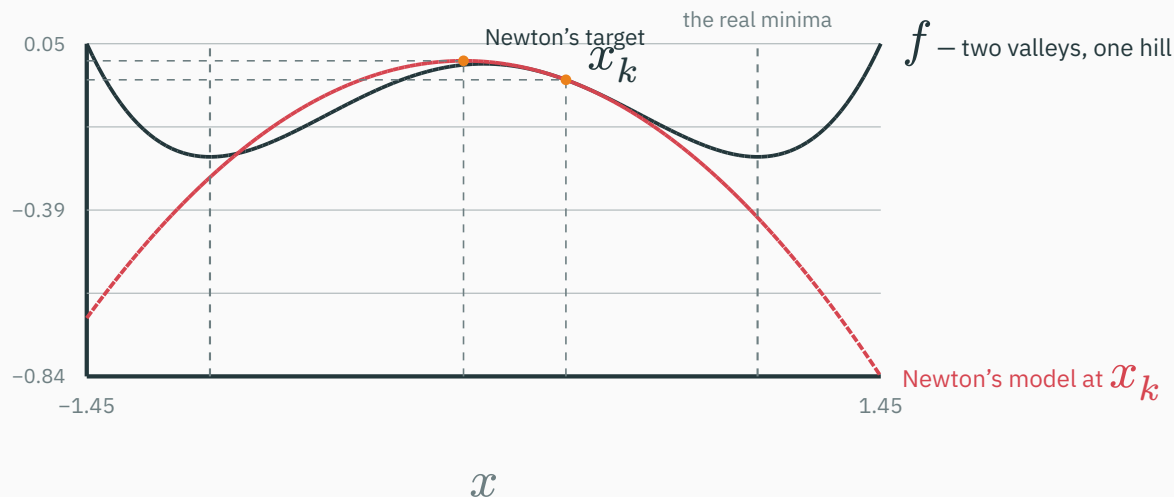
Newton · dives straight in



GD leaks along $-y$ and escapes downhill; Newton's model of a saddle is the saddle — its “jump to the stationary point” lands **on** it, in one step

Danger 2: negative curvature points it uphill

On a double well, stand at $x_k = 0.3$ where $f'' = -0.73 < 0$ — the parabola **frowns**:



a frowning parabola's stationary point is its **peak**: the “Newton step” here moves **left, toward the hilltop at $x = 0$** — and iterating converges to that local maximum

When H is not positive definite, $-H^{-1}\nabla f$ need not point downhill. Newton without a safety check is only trustworthy where the world is bowl-shaped.

The repair shop

All practical second-order methods are Newton plus a seatbelt:

- **Damping**: solve $(H + \tau I) \delta = -\nabla f$ — adding τI lifts every eigenvalue by τ , forcing the model convex (L5: eigenvalues shift together).
- **Line search**: take the Newton **direction**, but test the length before committing.
- **Trust region**: only believe the model inside a radius; grow or shrink the radius by results.

Hold the “ $+\tau I$ ” thought. In twenty minutes it returns wearing a famous name — and it’s the exact knob inside `curve_fit`.

Sums of squares: nonlinear least squares

Back to the sensor: the loss is a sum of squares

Our eight readings, and the model $\hat{y}(t) = a e^{-bt}$ with $\theta = (a, b)$:

t_i	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5
y_i	2.465	1.762	1.426	0.928	0.914	0.632	0.385	0.352

Per point, a **residual** — the model's miss; the loss adds the squares (L14: this is the Gaussian NLL):

$$r_i(\theta) = a e^{-bt_i} - y_i, \quad \mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^8 r_i(\theta)^2 = \frac{1}{2} \|\mathbf{r}(\theta)\|^2$$

The $\frac{1}{2}$ is cosmetics: it cancels the 2 that differentiation drops. This **shape** — squared residuals — is everywhere: calibration, GPS, camera bundle adjustment, regression.

The residual is a vector-valued map — L9's customer

$r : \mathbb{R}^2 \rightarrow \mathbb{R}^8$ — two parameters in, eight misses out. Its derivative is a **Jacobian** (L9): one row per residual, one column per parameter, 8×2 :

$$J_{i1} = \frac{\partial r_i}{\partial a} = e^{-bt_i}, \quad J_{i2} = \frac{\partial r_i}{\partial b} = -a t_i e^{-bt_i}$$

At the bad first guess $\theta_0 = (1.0, 0.2)$, three of the eight rows:

row	t_i	$\partial r_i / \partial a$	$\partial r_i / \partial b$
$i = 1$	0.0	1.000	0.000
$i = 2$	0.5	0.905	−0.452
$i = 8$	3.5	0.497	−1.738

Late-time rows barely feel a but feel b strongly — the Jacobian **is** the sensitivity table.

The gradient, in Jacobian clothes

Chain rule on $\mathcal{L} = \frac{1}{2} \mathbf{r}^\top \mathbf{r}$, shapes doing the bookkeeping (L9's habit):

$$\nabla \mathcal{L} = \underbrace{J^\top}_{2 \times 88 \times 1} \underbrace{\mathbf{r}}_{88 \times 1} \in \mathbb{R}^2$$

At $\theta_0 = (1.0, 0.2)$, with the measured residuals: $\nabla \mathcal{L} = J^\top \mathbf{r} = (-2.894, 0.937)$.

- Gradient descent is ready to go: $\theta \leftarrow \theta - \eta J^\top \mathbf{r}$. L17 machinery, nothing new.
- But Newton wants H — second derivatives of all eight r_i . Expensive... **unless the squares themselves hand us the curvature.** They do.

**Gauss-Newton: curvature
almost for free**

The idea: linearize the residuals, not the loss

Replace each residual by its L9 linear story at the current θ :

$$r(\theta + \delta) \approx r + J\delta \quad \Rightarrow \quad \mathcal{L}(\theta + \delta) \approx \frac{1}{2} \|r + J\delta\|^2$$

Look at what the right side **is**: a **linear least-squares problem in δ** — the one problem Module 1 taught us to solve exactly (and L16 proved convex).

A line inside a square is a parabola: linearizing r hands us a quadratic model of $\mathcal{L}$ — without ever touching a second derivative.

The Gauss-Newton step

Minimize $\frac{1}{2}\|r + J\delta\|^2$ over δ : its gradient is $J^\top(r + J\delta) = 0$ — the **normal equations** of the linearized fit:

Gauss-Newton: solve $(J^\top J) \delta = -J^\top r$, then $\theta \leftarrow \theta + \delta$; repeat.

Pattern-match against Newton's $H\delta = -\nabla f$:

- right side: $J^\top r = \nabla \mathcal{L}$ ✓ (last slide) — same gradient,
- left side: $J^\top J$ **standing where the Hessian stands**. Is it the Hessian? Almost —

Why this is secretly Newton

Differentiate $\nabla \mathcal{L} = J^\top r$ once more (product rule — each factor depends on θ):

$$H = J^\top J + \sum_{i=1}^n r_i \nabla^2 r_i$$

Gauss-Newton keeps the first term, **drops the second**. When is that fair?

- $r_i \approx 0$ near a good fit — the dropped term is weighted by the **misses**;
- or r_i nearly linear ($\nabla^2 r_i \approx 0$) — nothing to drop.

$J^\top J$ = curvature from **first derivatives only — and it's always PSD, so the model is never a frown. Newton's power, GD's ingredients.**

Numbers: the first step from $\theta_0 = (1.0, 0.2)$

Assemble the 2×2 system from the real data (all eight rows summed — check any entry by hand):

$$J^T J = \begin{pmatrix} 4.403 & -5.488 \\ -5.488 & 11.938 \end{pmatrix}, \quad J^T \mathbf{r} = \begin{pmatrix} -2.894 \\ 0.937 \end{pmatrix}$$

$$\begin{pmatrix} 4.403 & -5.488 \\ -5.488 & 11.938 \end{pmatrix} \boldsymbol{\delta} = \begin{pmatrix} 2.894 \\ -0.937 \end{pmatrix} \Rightarrow \boldsymbol{\delta} = \begin{pmatrix} 1.310 \\ 0.524 \end{pmatrix} \Rightarrow \boldsymbol{\theta}_1 = (2.310, 0.724)$$

From $(1.0, 0.2)$ straight to $(2.31, 0.72)$ — one solve of a 2×2 system.

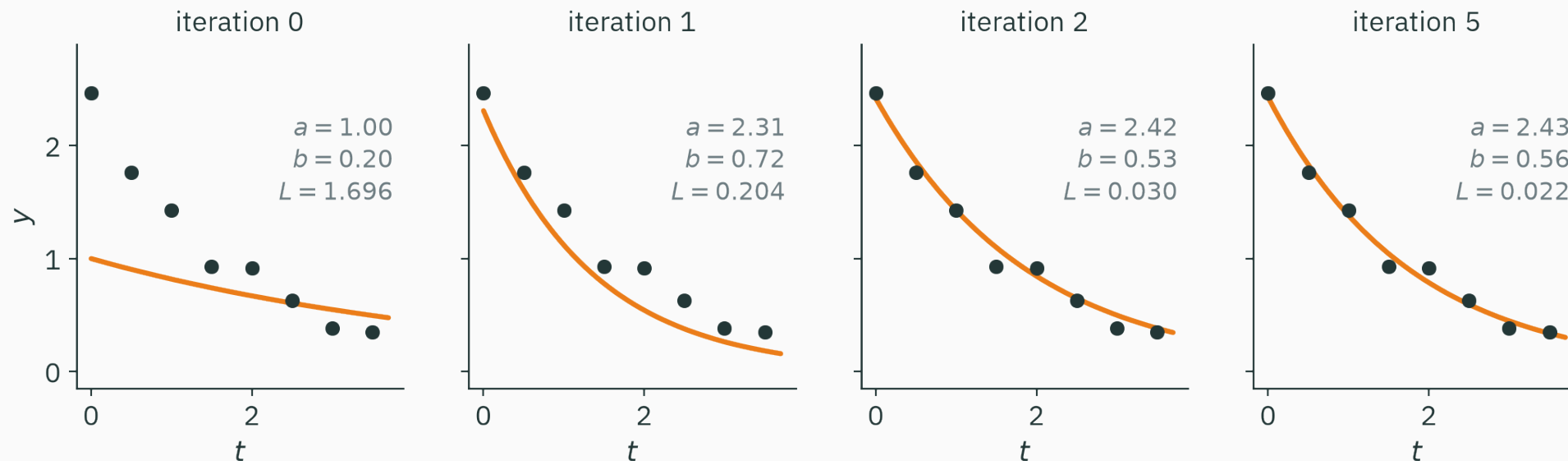
b **overshot** (true best: 0.563): the linearization was a rumor too, far from the fit.
No panic — iterate.

The run: five solves to machine precision

The whole optimization, on the real data (this table is the figure script's actual output):

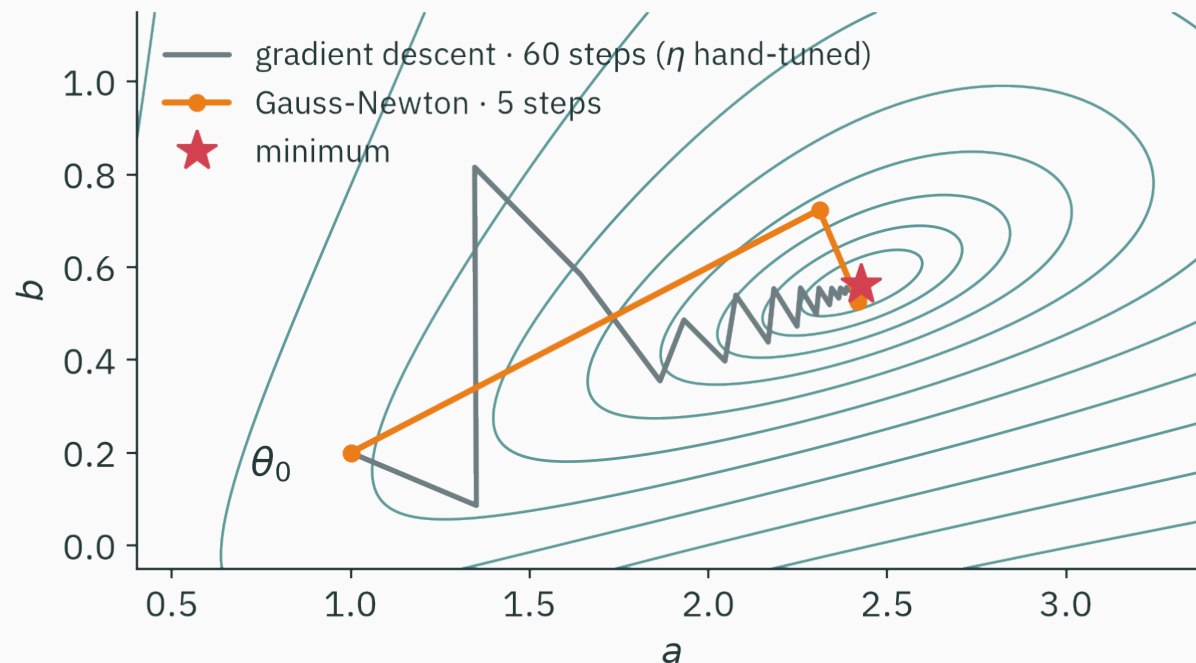
k	a_k	b_k	loss $\mathcal{L}(\theta_k)$
0	1.000000	0.200000	1.69626671
1	2.309903	0.723655	0.20392293
2	2.418643	0.525822	0.03006333
3	2.425973	0.560925	0.02159785
4	2.427402	0.562773	0.02158193
5	2.427436	0.562798	0.02158193

b rings — overshoot, undershoot, settle — as each linearization corrects the last. By $k = 5$ the loss has stopped changing at 8 decimals: the digits-double regime again.



Iteration 0 is a limp guess; one solve later the curve has snapped to the data; by 2 it's fitting **noise-level** detail. This is the six-step magic the hook promised.

The whole journey in the (a, b) plane



gradient descent zigzags into the curved valley — and its η was hand-tuned **using $J^\top J$'s eigenvalues**; 25% more η and it never converges. Gauss-Newton: two visible hops, three invisible ones.

Code: Gauss-Newton in eight lines

```
theta = np.array([1.0, 0.2])           # (a, b) – the bad guess
for _ in range(6):
    a, b = theta
    r = a * np.exp(-b * t) - y          # residuals           (8,)
    J = np.stack([np.exp(-b * t),      # dr/da               (8, 2)
                  -a * t * np.exp(-b * t)], axis=1)
    delta = np.linalg.solve(J.T @ J, -J.T @ r)
    theta = theta + delta
# theta = [2.427437, 0.562798]
```

Residuals, Jacobian, one small solve — no Hessian code anywhere, and no learning rate.

Levenberg–Marquardt: the trust dial

When Gauss-Newton stumbles

Gauss-Newton inherits Newton's temper, because it trusts a **model** built from a rumor:

- Far from the fit, the linearization lies — our very first step overshoot b by 29%.
- If the data barely constrains a parameter, $J^\top J$ is **nearly singular** (L6: tiny singular values) — the solve amplifies noise into a **wild** step.
- Nothing stops a step from **increasing** the loss. There is no seatbelt.

Section 5 already prescribed the seatbelt: **damp it** — add λI before solving.

One dial between two optimizers

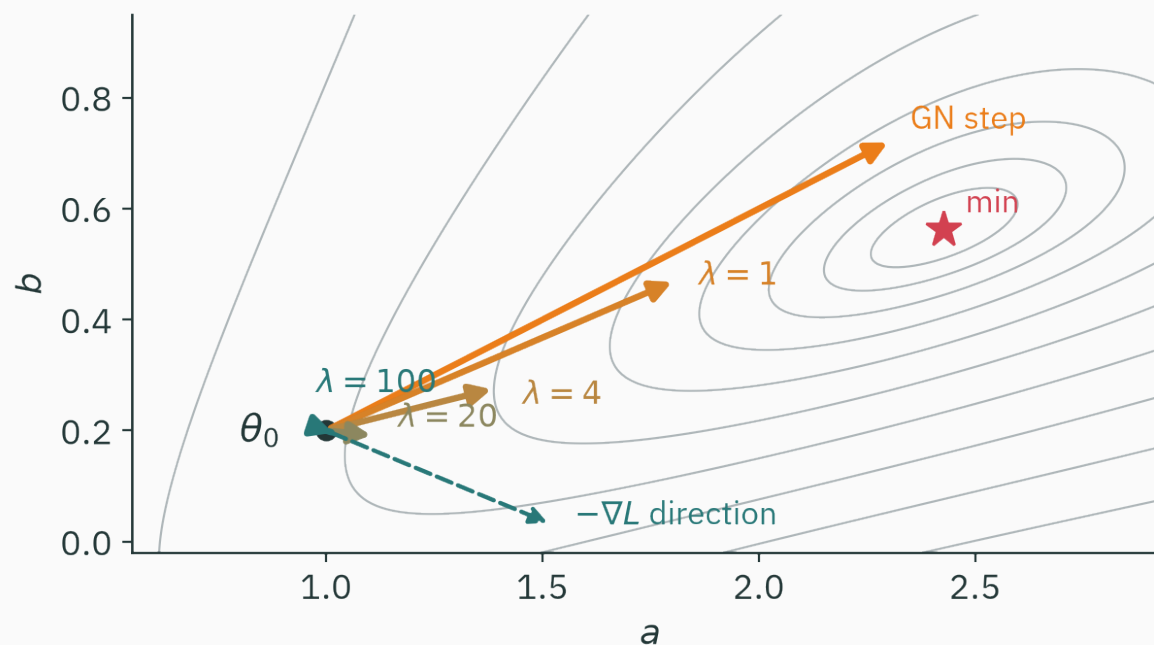
$$\text{Levenberg-Marquardt: } (J^\top J + \lambda I) \delta = -J^\top r$$

Turn the dial and watch who's in charge:

- $\lambda \rightarrow 0$: the damping vanishes — **pure Gauss-Newton**, full quadratic confidence.
- λ large: λI dominates $J^\top J$, so $\delta \approx -\frac{1}{\lambda} J^\top r = -\frac{1}{\lambda} \nabla \mathcal{L}$ — a **small gradient-descent step**: humble direction, tiny trust.

One scalar interpolates between “jump to the model's bottom” and “inch downhill”.

All steps solved from the same θ_0 on the real problem — only λ moves:



as λ climbs, the step **shrinks** and **swings** from the Gauss-Newton jump into the $-\nabla \mathcal{L}$ direction — confidence draining continuously out of the model

The λ policy: trust is earned

λ isn't tuned by you — it's adjusted **by results**, every iteration:

trial step...	action	meaning
lowered the loss	accept; $\lambda \downarrow (\div 10)$	model told the truth — trust it more, go more Newton
raised the loss	reject; $\lambda \uparrow (\times 10)$	model lied here — retreat toward small GD steps

A feedback controller with one state variable. Far away it behaves like safe GD; near the fit λ collapses and it **finishes** with Gauss-Newton's doubling digits.

This is Section 5's trust-region idea compressed into a scalar: λ small = big trust region, λ large = tiny one.

The hook, resolved

`scipy.optimize.curve_fit(model, t, y, method='lm')` runs exactly this loop — MINPACK's `lmdif`, Fortran from 1980, still fitting the world's sensors:

```
popt, pcov = curve_fit(model, t, y, p0=(1.0, 0.2))
# popt      = [2.427437, 0.562798]      <- curve_fit (LM)
# our GN k=5: [2.427436, 0.562798]      <- this lecture, 8 lines
```

Same answer to six decimals — on this easy, well-started problem LM's λ just stayed near zero and let Gauss-Newton drive.

- You never supplied a derivative: it builds J by **finite differences** (L10's tool, ideal at $d = 2$).

`curve_fit` = Gauss-Newton + the λ trust dial + finite-difference Jacobians. You have now built every piece.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizers-beyond

Race Newton and BFGS against gradient descent on 2-D losses: watch Newton teleport on the bowl, dive into the saddle, and go feral far from the minimum — then see damping tame it.

Try the ill-conditioned bowl first: it is this lecture's $\kappa = 50$ picture, live.

In Levenberg–Marquardt, cranking λ up makes the step approach...

A. the pure Gauss-Newton step

B. a tiny step along $-\nabla \mathcal{L}$

C. the exact Newton step (with the true Hessian)

D. a step of fixed length in a random descent direction

Correct: B — a tiny step along $-\nabla \mathcal{L}$

Why: For large λ , $(J^\top J + \lambda I) \approx \lambda I$, so $\delta \approx -\frac{1}{\lambda} J^\top r = -\frac{1}{\lambda} \nabla \mathcal{L}$: the **direction** becomes the negative gradient and the **length** shrinks like $1/\lambda$ — the dial controls both at once. (C is the other trap: LM never sees the true Hessian; even at $\lambda = 0$ it has only $J^\top J$, the second-derivative-free approximation.)

Summary & what's next

Lecture 18 — summary

- **Newton** ★: minimize the L9 quadratic model $\rightarrow$ solve $H\delta = -\nabla f$. A matrix-valued learning rate: step $1/\lambda_i$ per eigen-direction — κ is priced in.
- **On quadratics**: one step to the exact minimum, from anywhere, at any κ .
- **Near any minimum**: quadratic convergence — the error squares, digits double.
- **The bill**: d^2 memory, $O(d^3)$ solve; **the dangers**: saddles attract it, negative curvature sends it uphill. Seatbelts: damping $+\tau I$, line search, trust regions.
- **Least squares** $\mathcal{L} = \frac{1}{2}\|\mathbf{r}\|^2$: $\nabla \mathcal{L} = J^\top \mathbf{r}$, $H = J^\top J + \sum_i r_i \nabla^2 r_i$.
- **Gauss-Newton**: drop the residual term — $(J^\top J)\delta = -J^\top \mathbf{r}$: curvature from first derivatives; five solves fit our sensor.
- **Levenberg–Marquardt**: $+\lambda I$ — a results-driven trust dial from GN to GD. That's `curve_fit`.

Lecture 18 — summary

Read before L19 — MML §7.1.3 (context) · Solomon, *Numerical Algorithms*, Ch. 9 — the reference treatment of nonlinear least squares & Gauss-Newton. **T9** this week: Newton and Gauss-Newton by hand and in NumPy, on this lecture's exact data.

Practice problems

Try on paper; verify against `np.linalg.solve` and `curve_fit`.

1. Run one Newton step on $f(x) = x^2 - 4x + 3$ from $x_0 = 0$. Why is one step enough — and what does GD with $\eta = 0.1$ leave undone after 10 steps?
2. On $f(x, y) = x^2 + xy + y^2$ from $(3, -1)$: solve $H\delta = -\nabla f$ by hand and verify you land at $(0, 0)$.
3. For $f(x) = x - \ln x$, show the Newton update is $x_{k+1} = 2x_k - x_k^2$ and that $e_{k+1} = e_k^2$ exactly. How many steps from $x_0 = 0.5$ to 300 correct digits?
4. Run Newton on the double well $f(x) = x^4/4 - x^2/2$ from $x_0 = 0.3$ for three steps. Where is it converging, and what is the sign of f'' along the way?
5. For the model $\hat{y} = a \sin(bt)$: write r_i and both columns of J , then the Gauss-Newton system at a given (a, b) .
6. In LM, show $\delta(\lambda) \rightarrow -\frac{1}{\lambda} \nabla \mathcal{L}$ as $\lambda \rightarrow \infty$. Which **two** properties of the step does λ therefore control simultaneously?



Why deep learning still runs first-order

★ OPTIONAL

If Newton is this good, why is every LLM trained with Adam-flavored GD (L17 )?

per step, d parameters	GD/Adam	Newton
memory	$O(d)$ — a few copies of θ	$O(d^2)$: at $d = 10^9$, H is $\sim 4 \times 10^9$ GB
arithmetic	$O(d)$ past the gradient	$O(d^3) = 10^{27}$ FLOPs — years per step
minibatch noise	averages out (L17)	curvature estimated from noise — precision wasted
saddles (everywhere in high- d)	drifts past	attracted to them



Why deep learning still runs first-order

★ OPTIONAL

The workarounds are a research industry: **L-BFGS** (build a low-rank H^{-1} sketch from gradient history), Hessian-**vector** products Hv at gradient cost (L11 ) , and structured-curvature optimizers (K-FAC, Shampoo, Muon). Every one is a way to **respect** H without ever writing it down — Gauss-Newton's move, philosophically, all over again.

Newton doesn't step downhill — it teleports to the bottom of its local bowl.

Next: **Constrained Optimization: Lagrange Multipliers** — so far the search was free; real problems have fences.